

Apunte de la unidad 1: Protocolo Git y plataforma Github

Programación con Objetos 1



[Introducción a Git](#)

[Proyectos y repositorios](#)

[Repositorio local](#)

[Commit](#)

[Ramas \(*branches*\)](#)

[Generar un repositorio local](#)

[Ejemplo con repositorio local](#)

[Creación del proyecto y del repositorio local](#)

[Commits](#)

[Descartando cambios en el working directory](#)

[Descartando cambios ya commiteados](#)

[Repositorio remoto](#)

[Generar un repositorio remoto vacío en github](#)

[Subir los commits al repositorio remoto](#)

[Generar un repositorio local vinculado a un repositorio remoto ya creado](#)

[Subir los commits al repositorio remoto](#)

[Generar un repositorio local vinculado a un repositorio remoto nuevo a partir de github classroom.](#)

[Una persona que programa en 2 \(o más\) repositorios locales para el mismo repositorio remoto](#)

Introducción a Git

Git es una herramienta que resuelve, principalmente, los siguientes problemas:

- Versionado de código: te guarda la historia de tu código, por lo que podés volver a una versión previa. Es muy útil para programar sin miedo a romper: si rompí entonces puedo recuperar una versión previa que funcionaba y volver a intentar.
- Trabajo en equipo: Distintas personas pueden modificar código en simultáneo de un mismo proyecto e integrar los cambios de manera prolija.
- Resguardo del código: el código se puede guardar en distintos repositorios, en especial en alguno que está en la nube. Por lo que si se nos rompe la compu el proyecto no está perdido.

Estos tres problemas son comunes en el desarrollo de software. Git es hoy en día la herramienta más utilizada (y por lejos!) para resolverlo, pero no es la única. Mercurial surgió al mismo tiempo. SVN y CVS son predecesores de éstas tecnologías. Hoy nos enfocamos en git por su popularidad y utilidad. Las otras herramientas utilizan conceptos similares.

Este apunte evita intencionalmente ahondar en el uso de git para trabajo en equipo. Ese tema es suficientemente complejo como para ganarse un apunte propio. Aquí trabajaremos con los conceptos básicos de git y el uso para un proyecto unipersonal.

Proyectos y repositorios

Git no tiene el concepto de proyecto; la unidad en que se agrupa el código es **repositorio**. Por lo tanto, cada repositorio es para un proyecto en especial y se considera una mala práctica tener en el mismo repositorio archivos que corresponden a dos proyectos distintos.

Queda claro que 1 repositorio implica 1 proyecto, pero la relación inversa no se cumple: 1 proyecto no implica un único repositorio. Sino que puede haber muchos repositorios que repliquen los archivos del proyecto. Al trabajar en un proyecto, quien programa elige cuándo subir los cambios a un repositorio. Eso hace que los distintos repositorios del proyecto vayan divergiendo según el equipo del proyecto actúa sobre los mismos. Es también tarea del equipo sincronizarlos.

Repositorio local

La forma más simple de usar git es teniendo un único repositorio para el proyecto que se aloja en la máquina de desarrollo. Se le suele llamar repositorio **local**.

Commit

En este esquema el desarrollador trabaja en su ambiente y cuando considera que su código está correcto lo pone en el repositorio. El proceso de poner los cambios en el repo se realiza con un comando llamado: **commit**. Como bien argentinos nos gusta verbalizar las

palabras extranjeras y decimos *commitear* a la acción de ejecutar el comando y *commit* al conjunto de cambios que hemos decidido subir a nuestro repositorio.

Para cada *commit* se autogenera un identificador (un string). Este identificador permite obtener el proyecto en ese punto particular. Por ejemplo, si luego de haber hecho un *commit*, modifico el código, realizo un segundo commit y me doy cuenta que arruiné el proyecto, puedo volver a la versión que generó el primer commit y trabajar desde ese punto.

Una buena práctica es realizar commits cada vez que se agrega o modifica algo al proyecto y se considera que el proyecto está en un estado bueno. Puede estar incompleto pero la tarea que se planteó está resuelta. Dividir una tarea grande en tareas pequeñas para así realizar commits seguidos es una habilidad que se debe adquirir. Se debe pensar que todo código no *commiteado* es susceptible de ser perdido, y por lo tanto ocasionar un desperdicio de esfuerzo. *Commitear* es la manera de capitalizar ese trabajo y tener un buen *checkpoint* al cual regresar en caso de problemas.

Utilizar git sólo con el repo local resuelve el versionado de código, pero no las otras dos tareas que comentamos al principio: trabajo en grupo y backup.

Ramas (*branches*)

Es importante entender que los *commits* que se ponen en el repositorio no son *archivos*, son *cambios* con respecto a un estado anterior del proyecto. Por lo tanto, cuando solicitamos obtener los archivos al momento de un *commit* particular, en realidad lo que la herramienta hace es construirlos a partir del historial de commits que ocurrieron desde el inicio del proyecto. Esta secuencia de commits forman una rama (***branch***). Un *commit* tiene sentido dentro de una rama particular que permite regenerar la historia.

A partir de cualquier commit de una rama se puede generar una bifurcación: una nueva rama. Esta característica es muy útil para trabajar en equipo y la consideramos avanzada, por lo que queda fuera del scope de este documento. Sin embargo es necesario conocer el concepto de rama porque siempre estamos trabajando sobre una. Al comenzar el proyecto inicia también una rama principal que por defecto se llama **main**. En repositorios más antiguos puede ser que se llame **master**.

Generar un repositorio local

Git es una herramienta que tiene una interfaz basada en una consola que se ejecutan comandos, como bash, powershell o el simple cmd.

Muchas herramientas ofrecen interfaces visuales que por detrás invocan los mismos comandos que podemos hacer desde la consola.

Por eso para arrancar es necesario realizar la instalación según el sitio oficial: <https://git-scm.com/downloads>. y ejecutar la configuración de un nombre y mail que el sistema utilizará para asociar a cada nuevo commit.

```
git config --global user.name "Leo Gassman"
git config --global user.email "leonardo.gassman@unq.edu.ar"
```

El presente documento muestra las acciones para ser usadas por línea de comando, y también la alternativa para ser usada desde el IDE (visual studio code). Todos los comandos que utilizamos en este apunte están pensados para ser ejecutados en una consola bash, sobre linux. Para otros sistemas operativos los comandos pueden variar un poco.

Hay 3 maneras de generar un repositorio local:

1: usando el comando **git init** que genera un repositorio local vacío:

```
mkdir miProyecto  
cd miProyecto  
git init
```

2: Al crear un proyecto wollok con **wollok-cli**, ya se genera el repo local:

```
wollok init miProyecto  
cd miProyecto
```

3: usando el comando **git clone** que copia un repositorio que se aloja en otro lado, generalmente en la nube. Para lo cual nos deben proveer la url al repositorio de origen:

```
git clone https://github.com/miUser/miProyecto.git  
cd miProyecto
```

Estos mismos comandos se pueden ejecutar dentro del IDE, y los tres nos dejan con una carpeta de trabajo (*working directory*) listos para trabajar en nuestros proyectos. En el primer caso la carpeta estará vacía, en el segundo tendrá los archivos de ejemplo de wollok, y en el tercero dependerá de los archivos que ya tenga el repositorio de origen.

A partir de ahora, todos los comandos que se ejecutarán deberán estar en el path del proyecto. Esa carpeta es nuestro *working directory*. Dentro de éste están todos los archivos del proyecto y además, en una carpeta oculta, se encuentra el repositorio local.

Ejemplo con repositorio local

Creación del proyecto y del repositorio local

Se puede generar el proyecto junto con el repo y luego abrir el IDE:

```
wollok init miProyecto  
code miProyecto
```

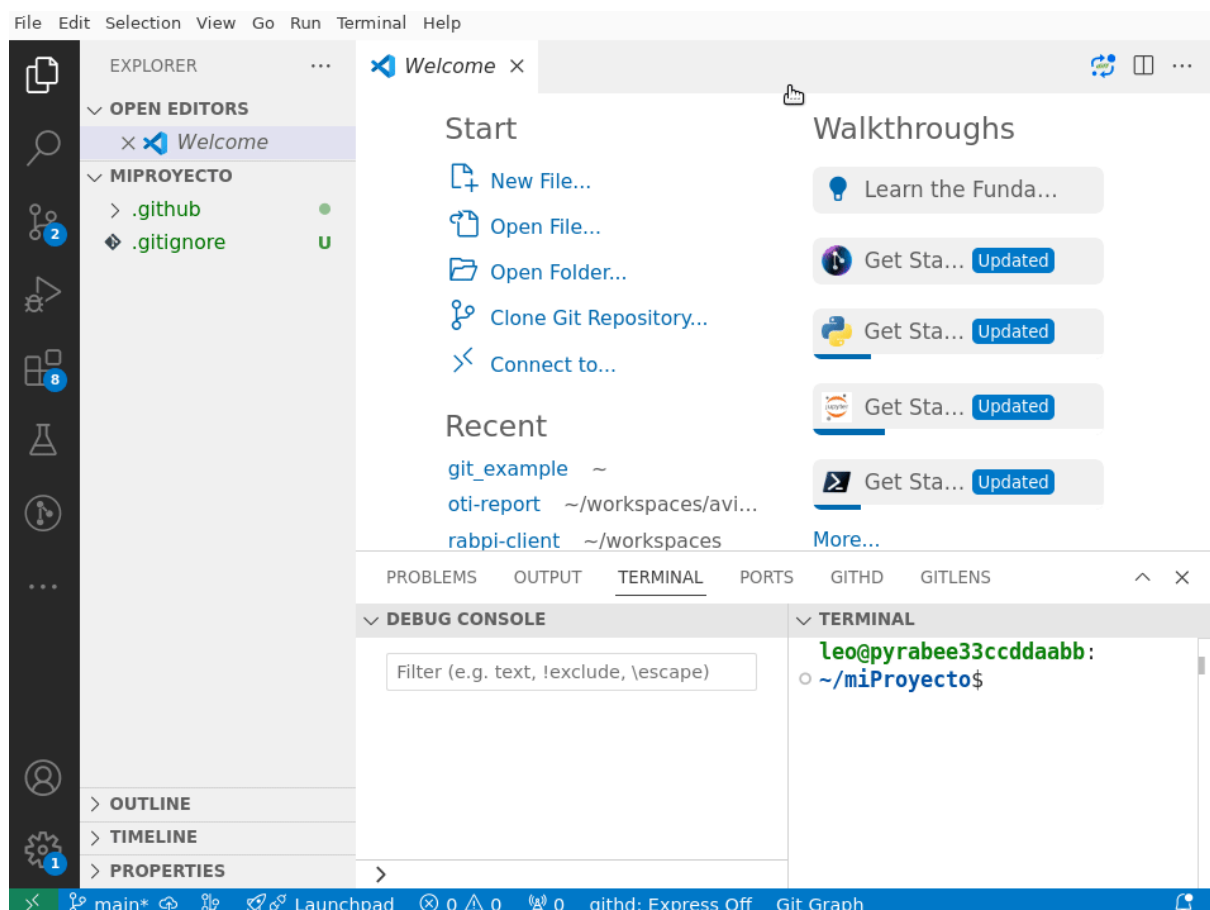
ó alternativamente crear la carpeta, abrir el IDE y crear el proyecto desde allí:

Nuestro primer ejemplo será utilizando el IDE, con el comando wollok-cli.

```
mkdir miProyecto  
code miProyecto
```

Una vez abierto el IDE:

1. control + shift + p
2. escribir: *Wollok: Generate a new Wollok Project in current folder* y luego apretar el enter



Commits

Independientemente de la manera en que generamos el proyecto, vemos al ingresar en el icono de source-control que aparecen las principales opciones para trabajar con el repo git

Todos los archivos del proyecto aparecen con la letra *U* de *untracked*, ya que son nuevos. Abajo a la izquierda se ve la rama (*branch*) actual: *main*. Si se desea ver esta información por consola, se utiliza el comando *git status*

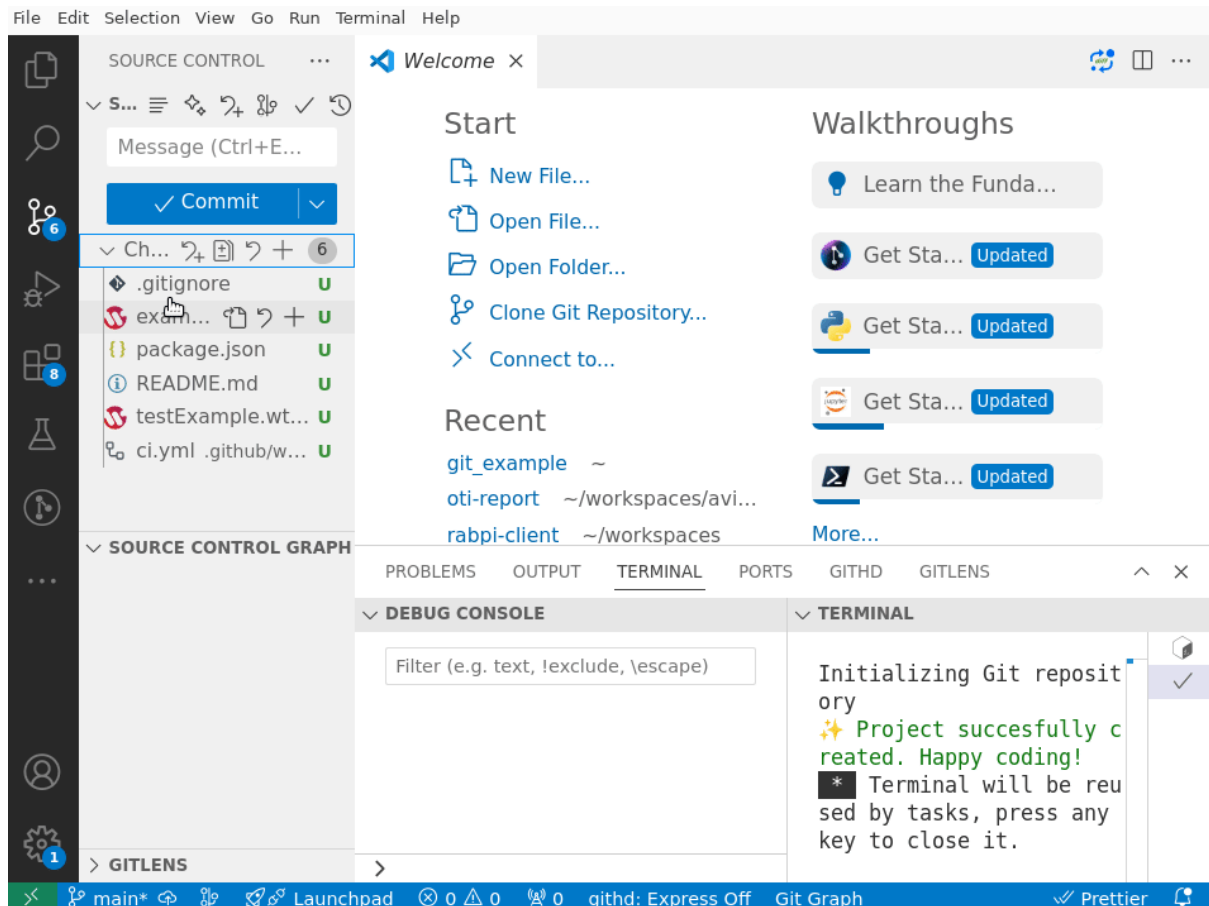
```
git status
```

Para realizar el primer *commit*, primero se deben agregar al **stage**. Ésta se puede interpretar como un área o índice donde están los archivos que tienen los cambios que queremos incluir en el próximo commit. Esto se puede hacer de varias maneras. por consola se puede usar ``git add nombreArchivo`` por cada archivo a incluir. Alternativamente se puede agregar todos los archivos nuevos o con cambios ejecutando ``git add -A``.

Luego, se debe ejecutar el comando *commit*. Éste siempre debe ir acompañado de un mensaje que describa el cambio realizado. En caso de no incluirlo en el comando, el sistema operativo abrirá un editor de texto donde obliga a escribir el mensaje. Al cerrar el editor se realiza el *commit*. Para evitar ese paso extra hay que utilizar el parámetro `-m` en el comando:

```
git add -A  
git commit -m "creación del proyecto"
```

Desde el IDE también se pueden agregar los archivos individualmente o agregar todo lo que tengan cambios. Luego realizar el commit incluyendo el mensaje:



A partir de ahora, se trabaja para modificar el código. Se pueden ir realizando múltiples commits. Esto va generando en la rama *main* la sucesión de *commits*. En el siguiente ejemplo agregamos un segundo *commit*. En el *source control graph* se muestra todo el historial de cambios (*commits*) sobre la rama actual (*main*), dejando el más nuevo en la parte superior.

The screenshot shows the VS Code IDE interface. On the left, the Explorer sidebar displays a project named 'MIPROYECTO' with files: '.github', 'log', '.gitignore', 'example.wlk' (selected), 'package.json', 'README.md', and 'testExample.wtest'. The editor window shows the code for 'example.wlk' with the following content:

```

1  object pepita {
2      var energia = 100
3
4      method energia() { return energia }
5
6      method volar(distancia) {
7          energia = energia - distancia - 10
8      }
9      method comer(comida) {
10         energia = comida.energia()
11     }
12 }
13

```

Below the editor, the 'TERMINAL' tab is active, showing the output of a test runner:

```

> group of tests for pepita
  ✓ "pepita has initial energy"

  ✓ 1 passed
  * Terminal will be reused by tasks, press any key to close it.

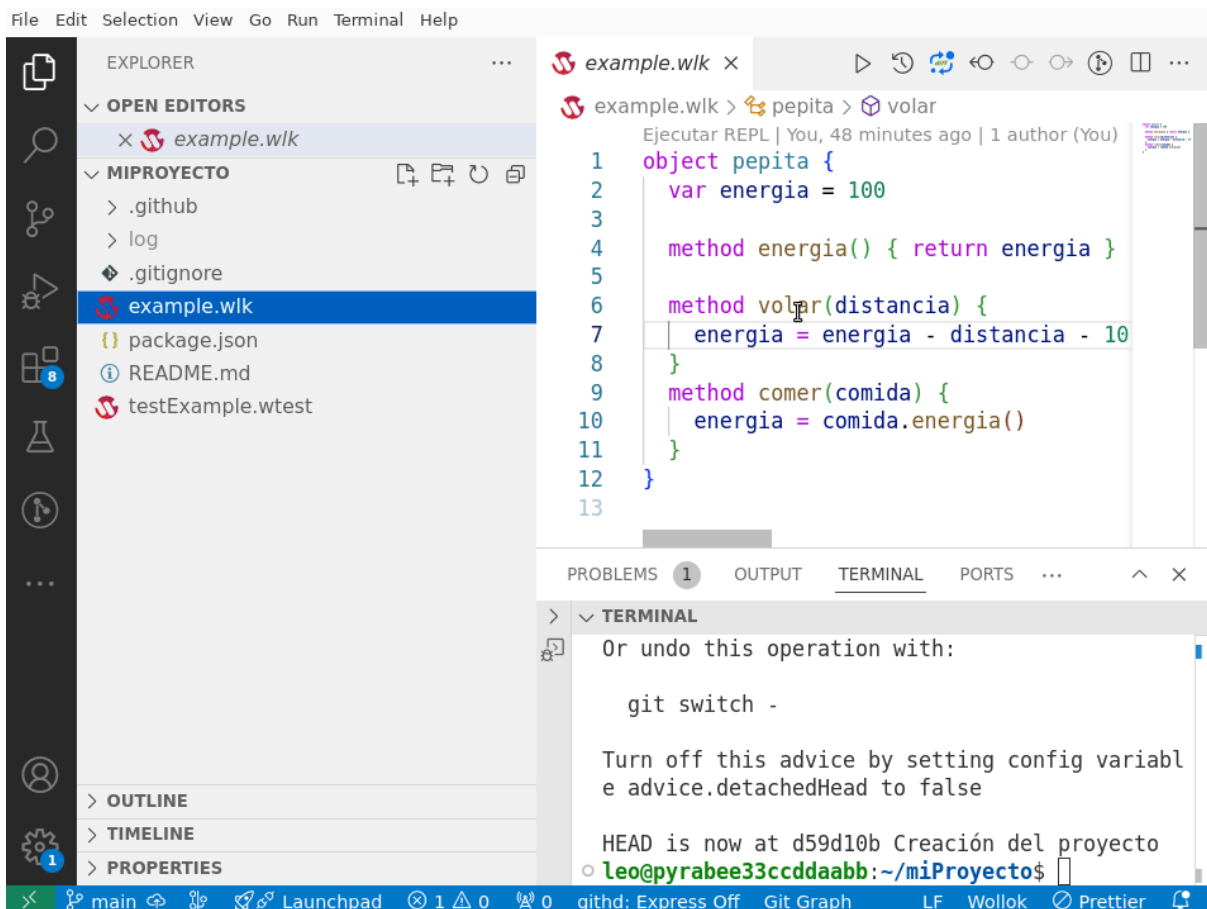
```

Descartando cambios en el *working directory*

Si realizamos cambios en el *working directory* que queremos descartar para volver a la última versión confiable (es decir, al último *commit*). Se puede utilizar

```
git reset --hard
```

Desde el IDE, esa acción se hace utilizando el botón *discard changes*:



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'MIPROYECTO' with files like '.github', 'log', '.gitignore', 'example.wlk', 'package.json', 'README.md', and 'testExample.wtest'. The 'example.wlk' file is open in the editor, showing the following code:

```

1  object pepita {
2    var energia = 100
3
4    method energia() { return energia }
5
6    method volar(distancia) {
7      energia = energia - distancia - 10
8    }
9    method comer(comida) {
10     energia = comida.energia()
11   }
12 }
13

```

Below the editor, the TERMINAL panel is active, displaying the following text:

```

> Or undo this operation with:
>
> git switch -
>
> Turn off this advice by setting config variable
> advice.detachedHead to false
>
> HEAD is now at d59d10b Creación del proyecto
> Leo@pyrabee33cddaabb:~/miProyecto$

```

Descartando cambios ya commiteados

En caso de realizar un commit con código que consideramos malo, se puede revertir utilizando *reset* con la opción del commit a revertir.

```
git reset --hard HEAD~1
```

HEAD~1 significa el último *commit*.

Esto mismo se puede lograr desde el IDE usando la opción `undo last commit`

The screenshot shows a code editor with the following JavaScript code in `example.wlk`:

```
1 object pepita {
2   var energia = 100
3
4   method energia() { return energia }
5
6   method volar(distancia) {
7     energia = energia - distancia - 10
8   }
9   method comer(comida) {
10    energia = comida.energia()
11  }
12 }
13
```

The terminal output shows the following commands and messages:

```
Turn off this advice by setting config variable
advice.detachedHead to false

HEAD is now at d59d10b Creación del proyecto
Leo@pyrabee33ccddaabb:~/miProyecto$ git reset
--hard
HEAD is now at aeef196 Pepita habla español
Leo@pyrabee33ccddaabb:~/miProyecto$
```

Repositorio remoto

La manera más simple de respaldar nuestro proyecto en la nube es utilizando un repositorio remoto que replique los cambios del repositorio local. En el repo local se configura la URL del repositorio remoto bajo un nombre, que por defecto es *origin*.

El espejado no es automático: quien programa elige en qué momento sincronizar ambos repositorios. La operación **push** sube los commits nuevos de la rama actual al remoto. Pero para lograrlo, primero debemos poder crear el repositorio remoto y configurarlo en el local.

La URL del repositorio remoto es provista por algún servicio que ofrece el alojamiento de un repositorio git. *Github* y *Gitlab* son los dos proveedores de este servicio más famosos, pero podría haber otros. Incluso muchas organizaciones deciden levantar sus propios servidores donde hospedarlos.

Tanto la creación del repositorio remoto, como la configuración en el repositorio local para que quede vinculado, dependen de la manera en que se haya comenzado a trabajar:

En nuestro ejemplo, hemos iniciado primero el repo local con *wollok init* o *git init*. Entonces corresponde generar un repositorio remoto vacío en el servicio, obtener la url del repositorio.

remoto y ejecutar un comando para realizar la vinculación. A partir de ese momento se pueden realizar commits.

Alternativamente, puede suceder que el repositorio remoto esté creado de antemano, y hayamos creado nuestro repo local clonando una URL. En ese caso la vinculación con el *origin* es automática y podemos ejecutar los *push* desde el primer commit.

Generar un repositorio remoto vacío en github

Para continuar nuestra línea de trabajo, vamos a utilizar un repositorio en github para alojar nuestro proyecto. Ésta es la manera necesaria para cuando comenzamos un proyecto desde cero en un repositorio local, que fue lo que realizamos en las secciones anteriores.

Un detalle a tener en cuenta, es que hay dos URLs: por un lado la URL para usar el navegador que nos permite administrar el repositorio remoto y navegar los archivos asociados. Y por otro, la url real del repo, que suele terminar en *.git* que es la que necesitamos configurar en el repo local.

Importante es elegir las opciones que dejan el repositorio vacío: no incorporar README.md ni licencia, porque eso generaría commits que estén en el repo remoto y no en el local, generando dificultades para sincronizarlo.

La siguiente imagen asume que ya se tiene una cuenta en github y se ha ingresado al sistema.

Notar como la URL del browser está relacionada pero no coincide con la URL que se termina copiando. Por otro lado, hay distintas urls según el protocolo de comunicación. Por simplicidad elegimos la versión https.

The screenshot shows a web browser with multiple tabs open, including 'fix close co', 'Your Repox', 'Connection', 'Nueva pest', 'Pika: cambi', 'Nueva pest', and 'connection'. The address bar shows 'github.com/lgassman'. The GitHub profile page for 'lgassman' is displayed, featuring a pixel art avatar of a person with glasses and a red 'S' on their chest. The profile name is 'Leo Gassman' with the handle 'lgassman'. Below the name, it says '9 followers · 1 following'. The location is listed as 'Argentina'. The 'Overview' tab is selected, showing '32 Repositories' and '2 Stars'. A blue notification banner at the top right says 'You unlocked new Achievements with private contributions! Show them off by including private contributions in your Profile in settings.' Below the profile, there are four 'Popular repositories' listed: 'intro-to-java' (Public, Java, 2 stars), 'educiaa_bluetooth_hm10' (Public, C, 1 star), 'embebidos-unq' (Public, Java), and 'gtk-osx-build' (Public, Python). Each repository has a brief description and a link to its details.

La vinculación del repo local se hace ejecutando el comando *git remote*.

```
git remote add origin https://github.com/lgassman/miProyecto.git
```

Alternativamente desde el IDE:

The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'MIPROYECTO' with files: '.github', 'log', '.gitignore', 'example.wlk' (selected), 'package.json', 'README.md', and 'testExample.wtest'. The main editor window displays the content of 'example.wlk':

```

1  object pepita {
2
3
4      method energia() { return energia }
5
6      method volar(distancia) {
7          energia = energia - distancia - 10
8      }
9      method comer(comida) {
10         energia = comida.energia()
11     }
12 }
13

```

At the bottom, the Terminal window shows the following output:

```

Turn off this advice by setting config variable
advice.detachedHead to false

HEAD is now at d59d10b Creación del proyecto
• leo@pyrabee33ccddaabb:~/miProyecto$ git reset
--hard
HEAD is now at aeef196 Pepita habla español
o leo@pyrabee33ccddaabb:~/miProyecto$

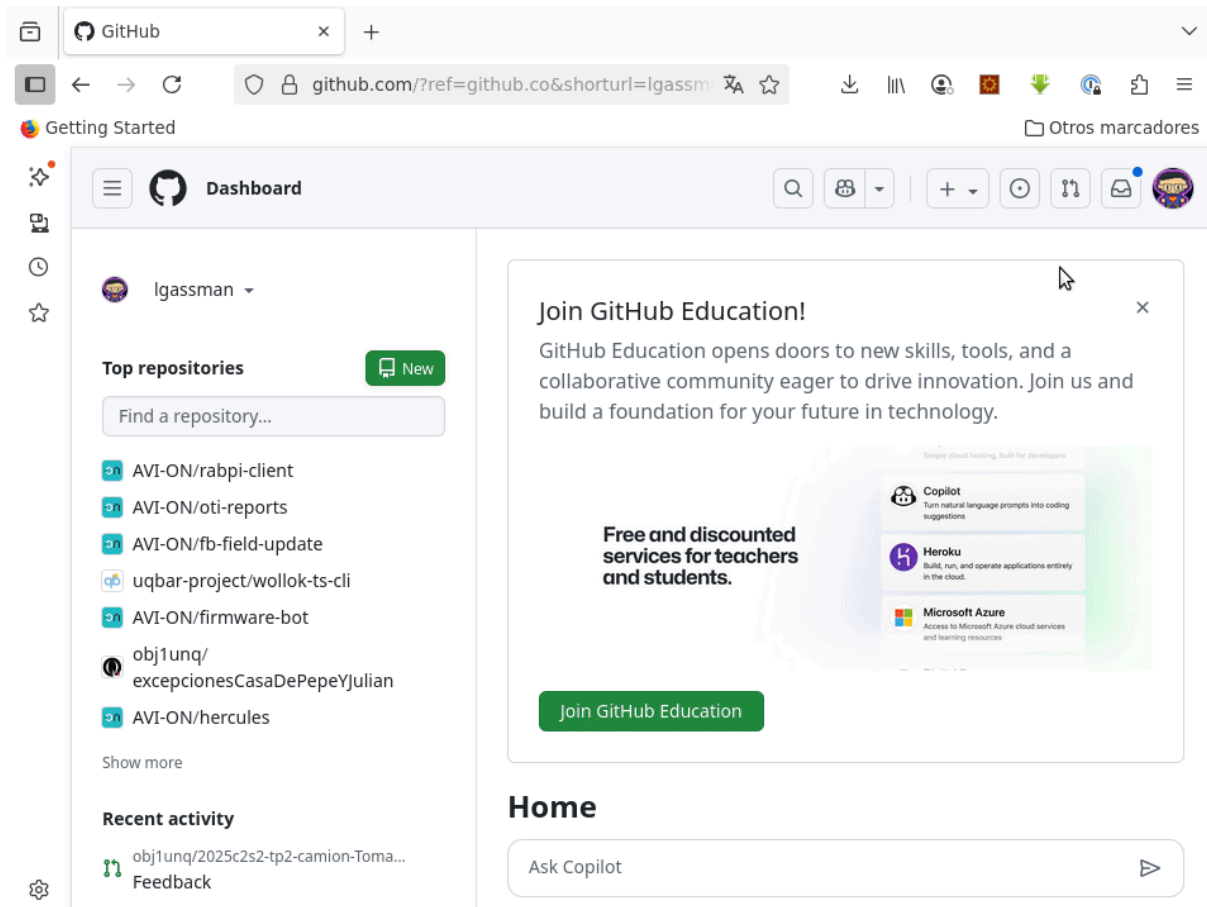
```

Prestar atención que la URL necesaria es la que finaliza en `.git` y que el nombre asociado sea *origin*. De llamarlo distinto va a complicar algunos comandos que usan *origin* por defecto.

Subir los commits al repositorio remoto

Para subir los *commits* del repositorio local al remoto, dejando sincronizado así ambos repositorios, se debe utilizar el comando *push*.

Es posible que al realizar el primer push nos exija loguearnos en github. Si se utiliza el IDE, éste abrirá una página para ingresar el usuario y el password de github. Si se realiza por línea de comando, también ocurrirá que se nos solicite el usuario y password, pero hay un detalle importante: github no permite el envío de la password real por este modo, lo que se debe hacer es generar un token desde los settings de desarrollador de la página que servirá a modo de password. Se puede utilizar el modo classic y agregar permisos de repo. El token generado debe ser guardado en algún archivo personal, ya que no se puede recuperar desde la página, pero sí se puede generar nuevos tokens.



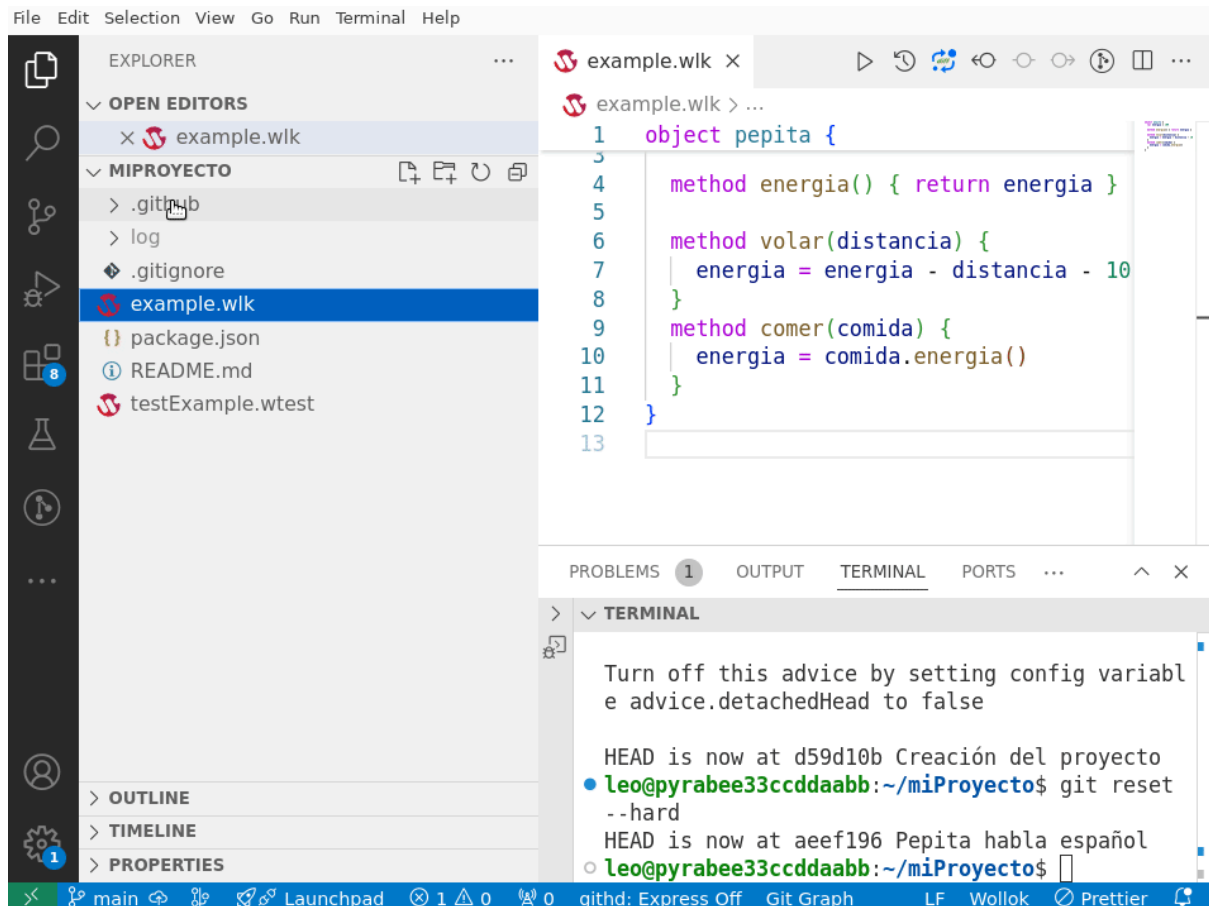
Posiblemente el primer push necesite una opción especial que implique publicar el branch main, ya que el mismo no existe actualmente en el repo remoto:

```
git push --set-upstream origin main
```

Luego, ya se puede utilizar el *push* sin parámetros para los próximos commits

```
git push
```

Desde el IDE aparece un botón especial para publicar el branch por primera vez:



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'MIPROYECTO' with files: '.github', 'log', '.gitignore', 'example.wlk' (selected), 'package.json', 'README.md', and 'testExample.wtest'. The main editor displays 'example.wlk' with the following code:

```

1  object pepita {
2
3
4      method energia() { return energia }
5
6      method volar(distancia) {
7          energia = energia - distancia - 10
8      }
9      method comer(comida) {
10         energia = comida.energia()
11     }
12 }
13

```

At the bottom, the Terminal panel shows the following output:

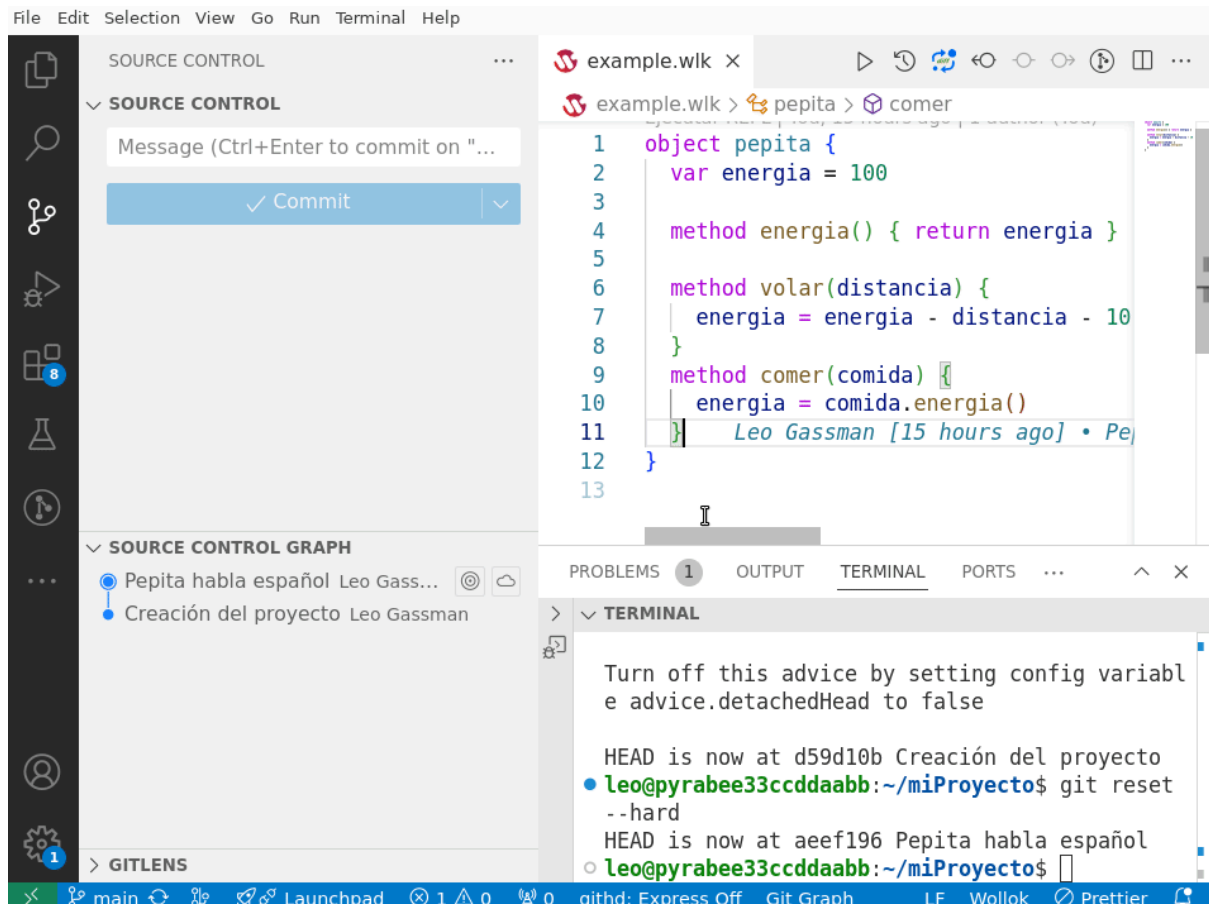
```

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at d59d10b Creación del proyecto
• leo@pyrabee33ccddaabb:~/miProyecto$ git reset --hard
HEAD is now at aeef196 Pepita habla español
o leo@pyrabee33ccddaabb:~/miProyecto$

```

A partir de ese momento los siguientes commits se pueden publicar utilizando el botón *synch changes* que incluye el push:



El IDE, en el *source control graph*, muestra con colores diferentes los commits que están publicados en el repositorio remoto de los que aún no.

Generar un repositorio local vinculado a un repositorio remoto ya creado

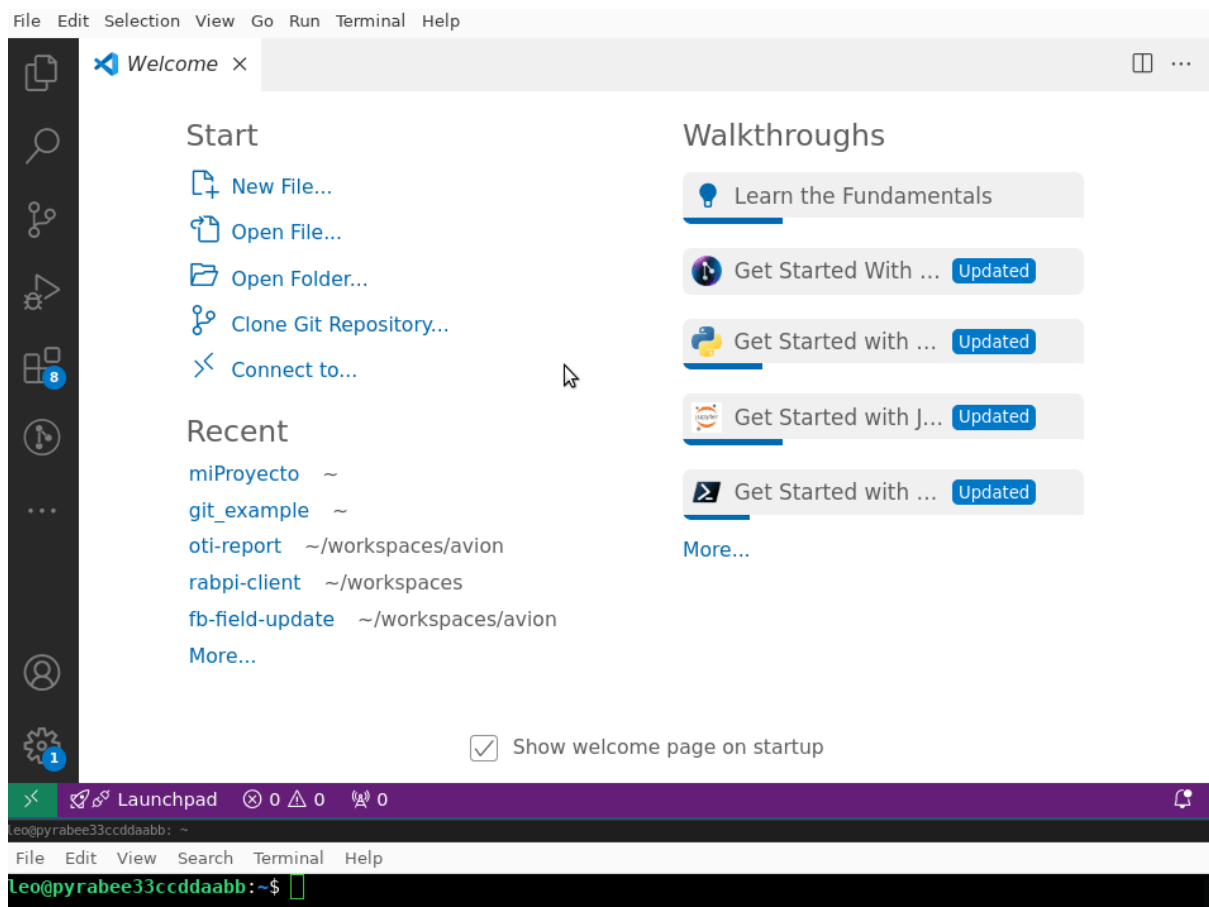
Si queremos comenzar a trabajar con un proyecto que ya tiene generado el repositorio remoto. Entonces no se debe iniciar el proyecto con *wollok init* o con *git init*. En su lugar generamos el repo local ya asociado al remoto y el *working directory* utilizando el comando *clone*. Para eso necesitamos que nos provean la URL (terminada en *.git*). En el caso de github, la URL se obtiene de la web.

```
git clone https://github.com/lgassman/miProyecto.git
code miProyecto
```

La versión desde el IDE es abriendolo sin una carpeta en especial.

```
code
```


Luego `control + shift + p` y escribiendo “Git: clone”. Se debe seleccionar la carpeta donde se generará el working directory.

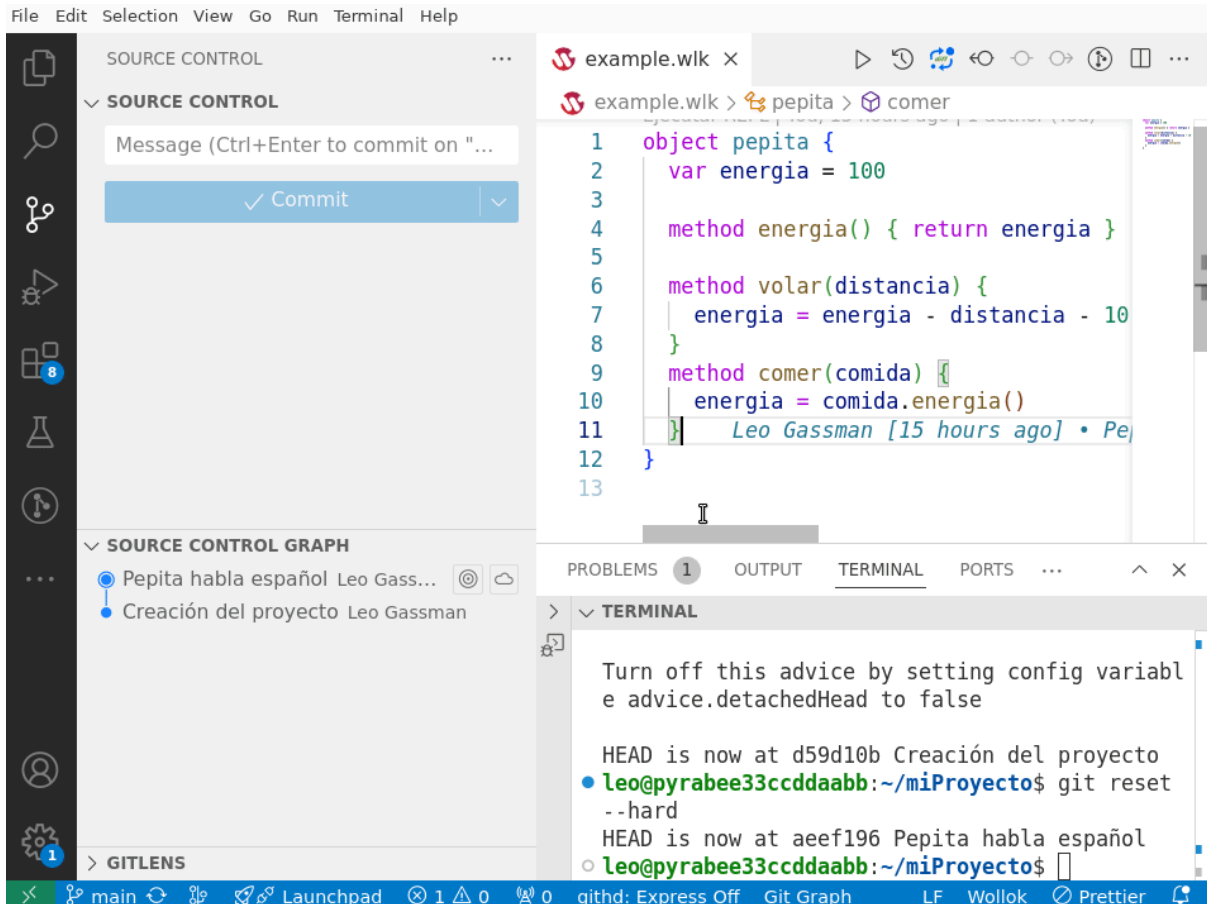


Subir los commits al repositorio remoto

Al clonar un repositorio ya existente, no es necesaria la etapa de publicación de la rama. Ya se puede realizar un push cuando haya un commit nuevo:

```
git push
```

Desde el IDE se utiliza el botón *sync changes* directamente, lo que incluye el push.



The screenshot shows an IDE interface with the following components:

- Source Control Panel (Left):**
 - Message (Ctrl+Enter to commit on "...")**: A text input field.
 - Commit**: A button to commit the changes.
 - SOURCE CONTROL GRAPH**: A list of commits:
 - Pepita habla español Leo Gass...
 - Creación del proyecto Leo Gassman
 - GITLENS**: A section at the bottom of the source control panel.
- Editor (Center):**
 - File: `example.wlk`
 - Content: A JavaScript object definition for `pepita`:


```
1 object pepita {
2   var energia = 100
3
4   method energia() { return energia }
5
6   method volar(distancia) {
7     energia = energia - distancia - 10
8   }
9   method comer(comida) {
10    energia = comida.energia()
11  }
12 }
13
```
- Terminal (Bottom Right):**
 - Output:


```
Turn off this advice by setting config variable
advice.detachedHead to false

HEAD is now at d59d10b Creación del proyecto
leo@pyrabee33ccddaabb:~/miProyecto$ git reset
--hard
HEAD is now at aeef196 Pepita habla español
leo@pyrabee33ccddaabb:~/miProyecto$
```

Generar un repositorio local vinculado a un repositorio remoto nuevo a partir de github classroom.

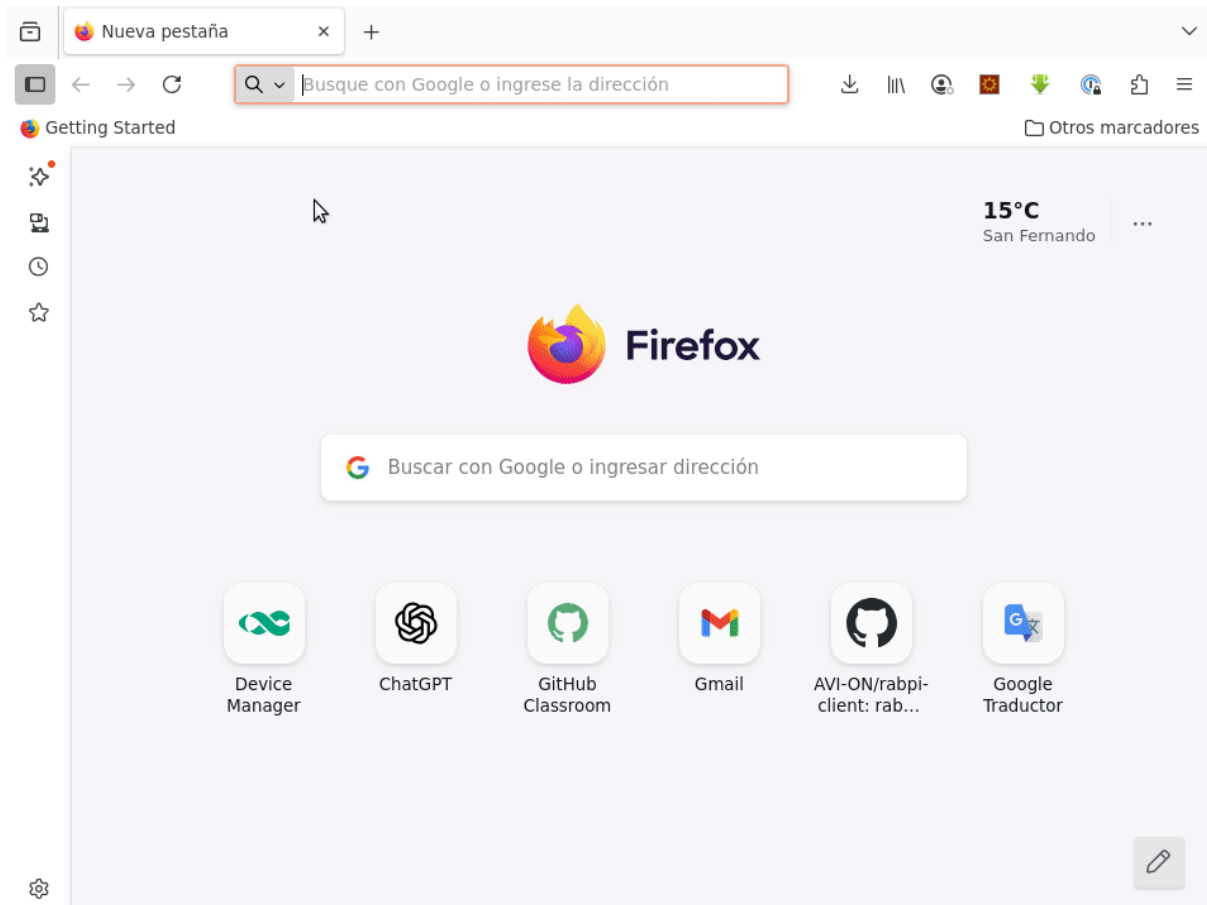
Github classroom es una herramienta que soluciona un problema puramente del dictado de una materia: El equipo docente tiene un repositorio remoto con un proyecto sobre el cual quiere que los y las estudiantes trabajen. En nuestro caso usamos proyectos en wolok cuyo enunciado se encuentra en el archivo README.md.

Ese repositorio remoto no puede ser utilizado directamente por los/las estudiantes, ya que los commits estarían interfiriendose. Lo que se necesita es que cada persona tenga su propio repositorio remoto, generado a partir del código del repositorio remoto del equipo docente.

Github classroom permite eso: generar por cada estudiante un repositorio remoto. El docente publica un link de un "assignment" (tarea). La primera vez que una persona use ese link en un browser le genera su propio repositorio remoto en github. Si se utiliza nuevamente, el link llevará a la página de github del repositorio creado previamente.

Una vez allí, se debe obtener el link del repositorio remoto (el terminado en .git) y clonarlo según se vio en la sección anterior.

La primera vez que se utiliza, además hay que elegir el nombre de quien programa de una lista previamente cargada por el equipo docente, lo que permite dar trazabilidad sobre los repositorios a lo largo de la cursada.



Una persona que programa en 2 (o más) repositorios locales para el mismo repositorio remoto

Una situación normal cuando trabajamos de manera individual es tener dos puestos de trabajos distintos para la misma persona: Por ejemplo, la computadora personal de la casa y la computadora del aula. La persona realiza tareas en su computadora, luego quiere continuar en el aula, y finalmente, seguir en su casa. En cada puesto de trabajo existirá un repositorio local generado de alguna manera como se trató en las secciones anteriores, probablemente clonando la misma URL en cada computadora. Es importante notar que es la misma persona quien trabaja, por lo tanto cuando se usa un puesto de trabajo, el otro está en desuso.

- El comienzo del trabajo es sin dificultades, tal cual lo hemos visto anteriormente. Se clona el repositorio en el primer puesto de trabajo (por ejemplo el aula) y se realizan commits. **Antes de cambiar de puesto se debe realizar commit y push.** De no hacerlo podría haber conflictos que queremos evitar.
- Al cambiar de computadora por primera vez, el repo es clonado nuevamente. Los *commits* anteriores, que fueron pusheados desde el aula, aparecen en la historia sin problema. Se pueden realizar nuevos *commits*. **Al terminar es muy importante realizar el último commit y el push antes de volver a cambiar de puesto de trabajo.** El *working directory* queda sin cambios.
- Al volver al aula, no se realiza un nuevo *clone*, ya que el ambiente de trabajo está creado. La diferencia es que por primera vez, vemos que el repo remoto está más avanzado que el repo local, ya que en el remoto están los commits desarrollados en la casa. Por eso, **siempre que empecemos a trabajar en un proyecto que fue clonado previamente, se deben descargar los commits que hay en el repo remoto, con una operación que denominamos pull.** Si se comienza a trabajar sin haberlo realizado podría haber conflictos. Al terminar, nuevamente se debe realizar commit y push.

El proceso puede resumirse como: **sincronizar el working directory, el repositorio local y el remoto al comenzar y al finalizar la tarea de programación.**

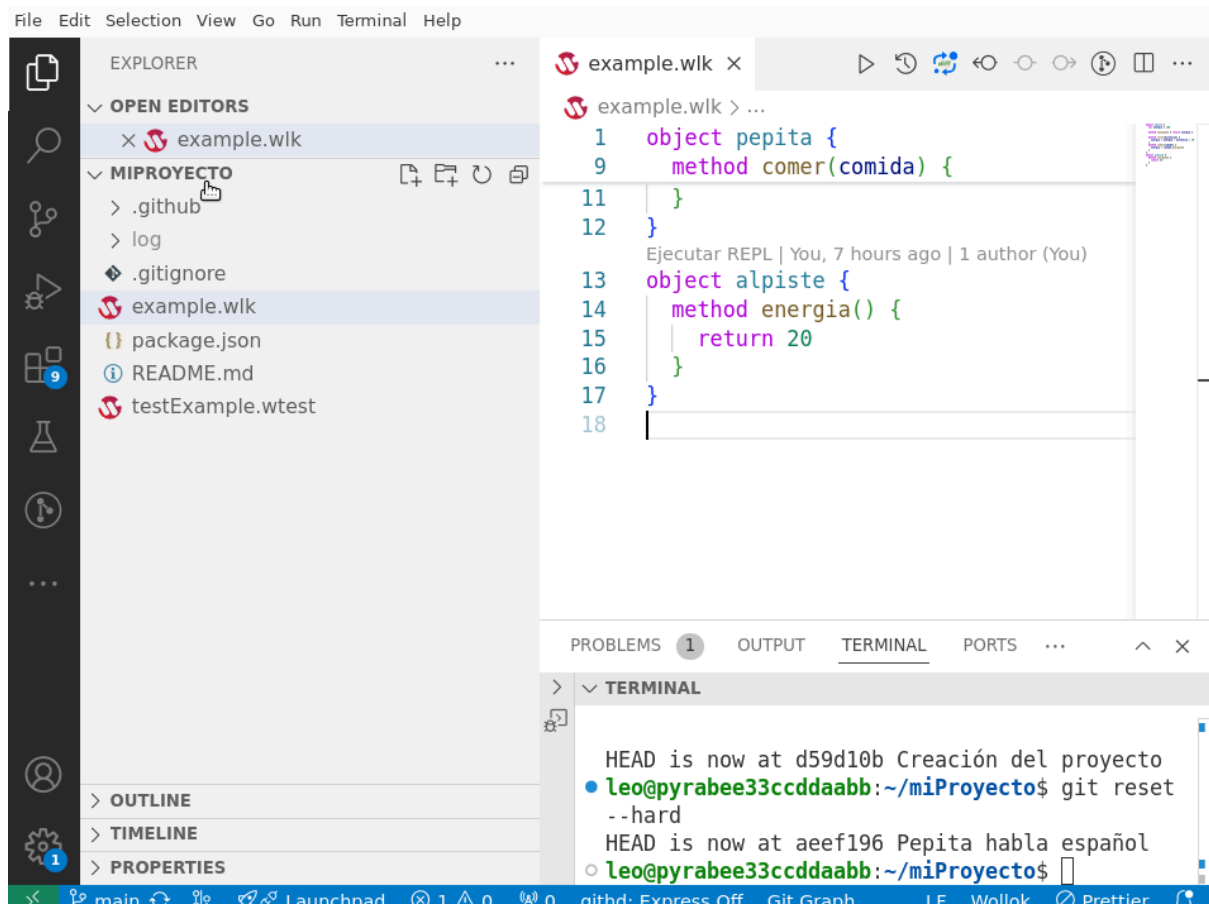
Un **pull** es una operación que incluye en realidad dos operaciones que pueden verse por separado: un **fetch** y un **merge**:

- La operación **fetch** baja los commits de todas las ramas que no están en el repositorio local (ya que podrían haberse introducido desde otro repositorio local de otra computadora). Estos nuevos commits no modifican la rama actual, ni cambia el directorio de trabajo. Simplemente trae información del remoto y las deja guardadas en unas ramas locales que se llaman de seguimiento. En nuestro modelo de trabajo solo utilizamos una rama: *main*, y un repo remoto: *origin*. Entonces en el repositorio local existen 2 ramas: *main* (donde trabajamos) y *origin/main* donde se tiene toda la información de seguimiento que bajamos del repo remoto.
- La operación **merge** agrega a una rama, los *commits* que están en otra rama. En nuestro caso, en la rama *main* se incorporan los commits de *origin/main*. Si fuimos prolijos y mantuvimos la buena práctica de sincronizar antes y después de trabajar con cada repositorio local, la rama *origin/main* contiene todos los *commits* de la rama *main* y los *commits* posteriores realizados en el otro puesto de trabajo. Recordando que un *commit* significa un cambio a partir de un estado anterior, la herramienta no tendrá ninguna dificultad en agregar los commits nuevos a la rama *main*. Además, actualiza el *working directory* dejándolo listo para trabajar.

Para realizar un pull:

```
git pull
```

Con el IDE se puede realizar el pull desde el menú:



```

1  object pepita {
9    method comer(comida) {
11  }
12 }
13
14 object alpiste {
15   method energia() {
16     return 20
17   }
18 }
  
```

```

HEAD is now at d59d10b Creación del proyecto
leo@pyrabee33ccddaabb:~/miProyecto$ git reset --hard
HEAD is now at aef196 Pepita habla español
leo@pyrabee33ccddaabb:~/miProyecto$
  
```

Conflictos

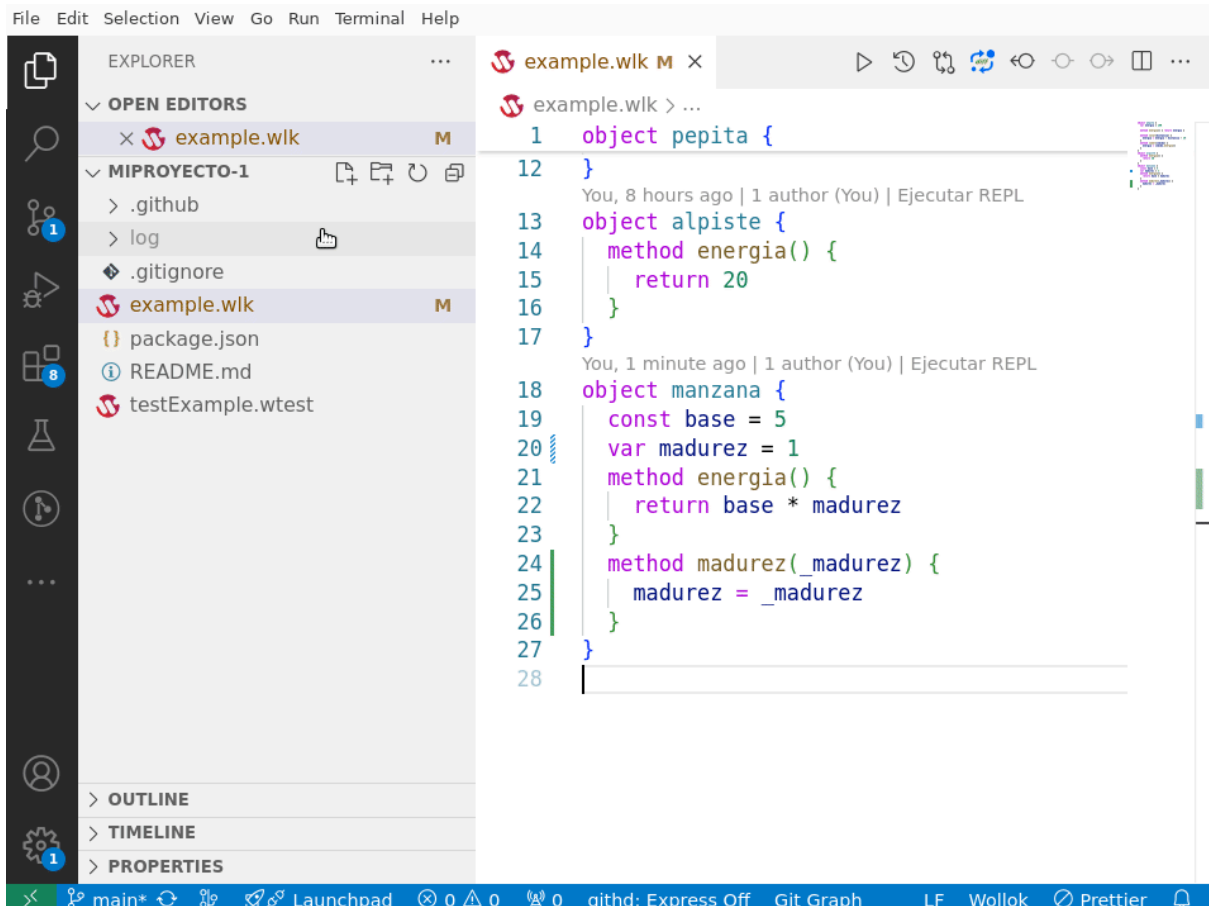
Como los humanos somos falibles, probablemente ocurra que trabajemos sin habernos sincronizado antes. Si la historia del repositorio remoto tiene commits que no fueron incorporados al local. Al momento de querer *pushear* nuevos cambios git se va a encontrar que los *commits* nuevos están pensados para ser aplicados a partir de un estado que no es el actual en el repositorio remoto. Se rechazará el *push* con un mensaje solicitando se realice el *pull* primero.

Al hacer el *pull*, se ejecuta el *fetch*, actualizando el *origin/main*. En el momento del *merge* git intentará mezclar los commits de ambas ramas, pero si detecta que se ha trabajado sobre las mismas líneas de código de los mismos archivos, no podrá resolverlo automáticamente, marcando que hay conflicto.

En el *working directory* dejará marcado entre picos “<<< == >>>” Las porciones de código conflictivas de cada rama. Marcará como HEAD el main y con el hash del commit conflictivo lo que quiso incorporar y no pudo.

Es responsabilidad de quien programa modificar el código fuente para resolver el conflicto. Luego de lo cual, se debe realizar un nuevo *commit* que unifique las diferencias. Luego de este proceso se puede realizar el *push* sin problemas.

En el ejemplo de la siguiente animación, Existía un commit “*madurar*” realizado en otra estación de trabajo y *pusheado* en el repositorio remoto. Mientras que en el repositorio local se trabajó en el *setter de madurez*. Al usar el botón *sync changes* se rechazó el push y se realizó el pull para traer los *commits* del repositorio remoto que no están en el local. En ese momento se puede ver en el *source control graph* como la rama *main* y *origin/main* han divergido. Se ve la lista de archivos con problema a la izquierda, y en el código fuente queda la mezcla de ambas versiones. Se resuelve a mano (en este caso ambos códigos debían ser necesarios). Se realiza el *commit* que unifica ambas historias. Finalmente se realiza el pull a través del botón *sync changes* nuevamente



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'MIPROYECTO-1' with files: '.github', 'log', '.gitignore', 'example.wlk' (marked with a red Git icon), 'package.json', 'README.md', and 'testExample.wtest'. The main editor area shows the 'example.wlk' file with the following JavaScript code:

```

1  object pepita {
12 }
13 You, 8 hours ago | 1 author (You) | Ejecutar REPL
14 object alpiste {
15   method energia() {
16     return 20
17   }
18 }
19 You, 1 minute ago | 1 author (You) | Ejecutar REPL
20 object manzana {
21   const base = 5
22   var madurez = 1
23   method energia() {
24     return base * madurez
25   }
26   method madurez(_madurez) {
27     madurez = _madurez
28   }

```

The status bar at the bottom indicates the current branch is 'main*' and shows various icons for Git, Launchpad, and other extensions.